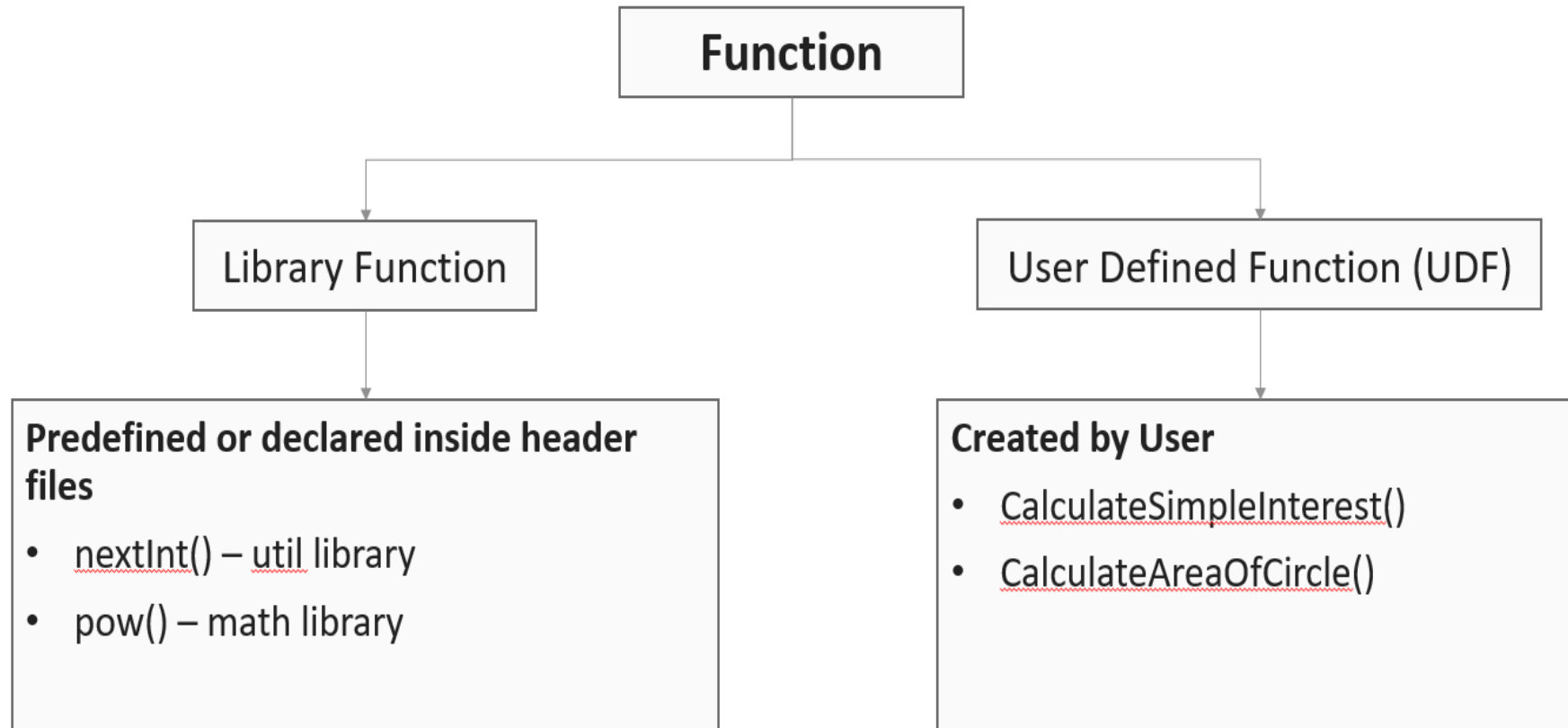


What is Method?

- ▶ A **method** is a group of statements that performs a **specific task**.
- ▶ A large program can be **divided** into the basic building blocks known as **method/function**.
- ▶ The function contains the set of programming statements enclosed by { }.
- ▶ Program execution in many programming language starts from the **main** function.
- ▶ **main** is also a **method/function**.

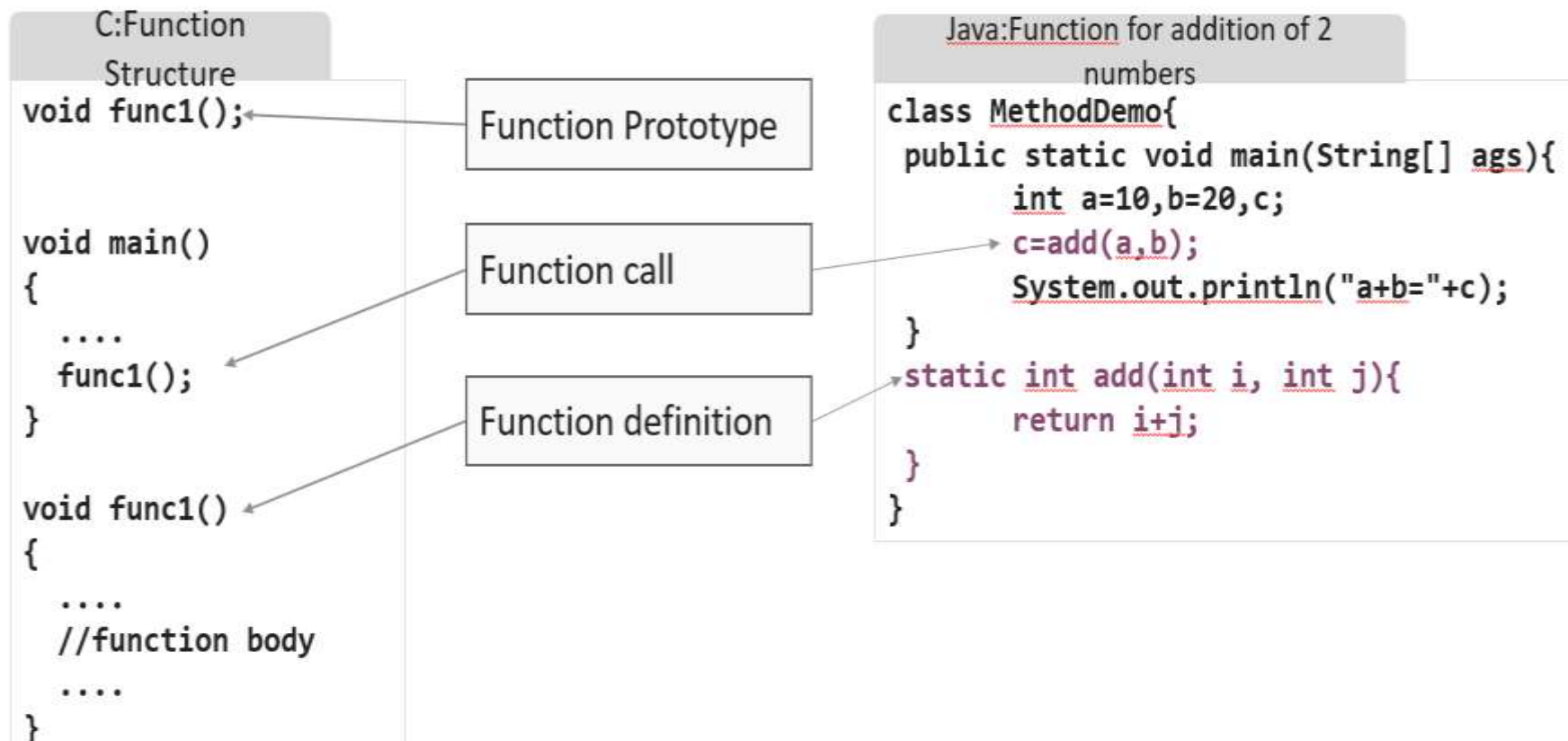
```
void main()  
{  
    // body part  
}
```

Types of Function



Program Structure of Function

- ▶ User-defined function's program structure is divided into three parts as follows:



Method Definition

- ▶ A method definition defines the **method header** and **body**.
- ▶ A **method body** part defines method logic.
 - ↳ Method statements

Syntax

```
return-type method_name(datatype1 arg1, datatype2 arg2, ...)  
{  
    functions statements  
}
```

Example

```
int addition(int a, int b);  
{  
    return a+b;  
}
```

WAP to add two number using add(int, int) Function

```
1. class MethodDemo{
2.     public static void main(String[] args) {
3.         int a=10,b=20,c;
4.         MethodDemo md=new MethodDemo();
5.         c=md.add(a,b);
6.         System.out.println("a+b="+c);
7.     }//main()
8.     int add(int i, int j){
9.         return i+j;
10.    }
11. }
```

Output:

a+b=30

Actual parameters v/s Formal parameters

- ▶ Values that are passed from the calling functions are known **actual parameters**.
- ▶ The variables declared in the function prototype or definition are known as **formal parameters**.
- ▶ Name of formal parameters can be **same** or **different** from actual parameters.
- ▶ Sequence of parameter is **important**, not name.

Actual Parameters

```
int a=10,b=20,c;  
MethodDemo md=new MethodDemo();  
c=md.add(a,b);  
// a and b are the actual parameters.
```

Formal Parameters

```
int add(int i, int j)  
{  
    return i+j;  
}  
// i and j are the formal parameters.
```

Return Statement

- ▶ The function can **return only one value**.
- ▶ Function **cannot** return more than one value.
- ▶ If function is not returning any value then return type should be **void**.

Actual Parameters

```
int a=10,b=20,c;  
MethodDemo md=new MethodDemo();  
c=md.sub(a,b);  
// a and b are the actual parameters.
```

Formal Parameters

```
int sub(int i, int j)  
{  
    return i - j;  
}  
// i and j are the formal parameters.
```

WAP to calculate the Power of a Number using method

```
import java.util.*;
public class PowerMethDemo1{
public static void main(String[] args){
    int num, pow, res;
    Scanner sc=new Scanner(System.in);
    System.out.print("enter num:");
    num=sc.nextInt();
    System.out.print("enter pow:");
    pow=sc.nextInt();
    PowerMethDemo1 pmd=new
                                PowerMethDemo1();

    res = pmd.power(num, pow);
    System.out.print("ans="+res);
} //main()

int power(int a, int b){
    int i, r = 1;
    for(i=1; i<=b; i++)
    {
        r = r * a;
    }
    return r;
} //power()
} //class
```

Output:

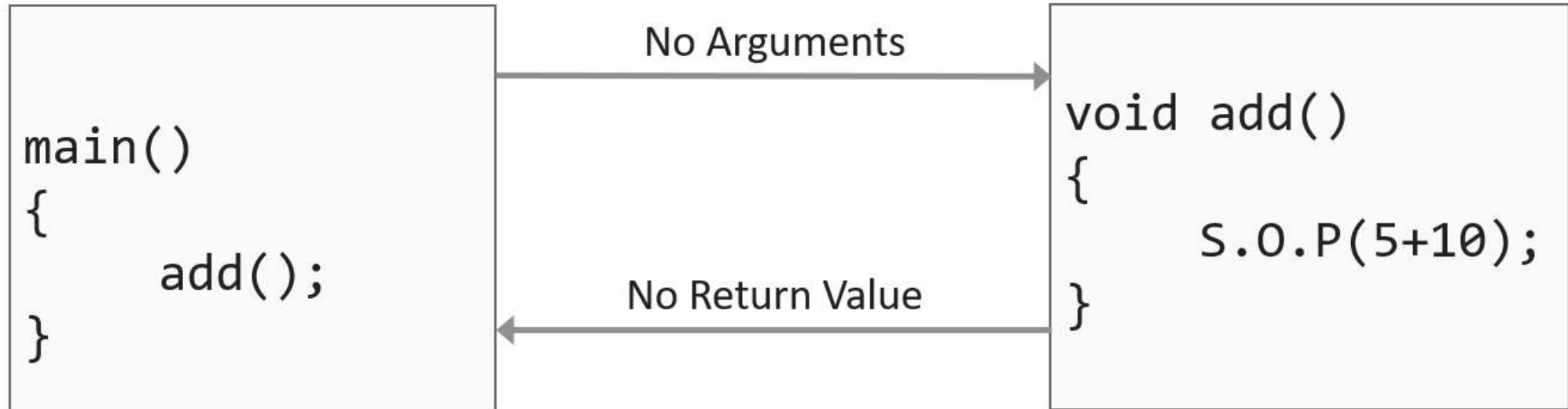
```
enter num:5
enter pow:3
ans=125
```


Types of Methods(Method Categories)

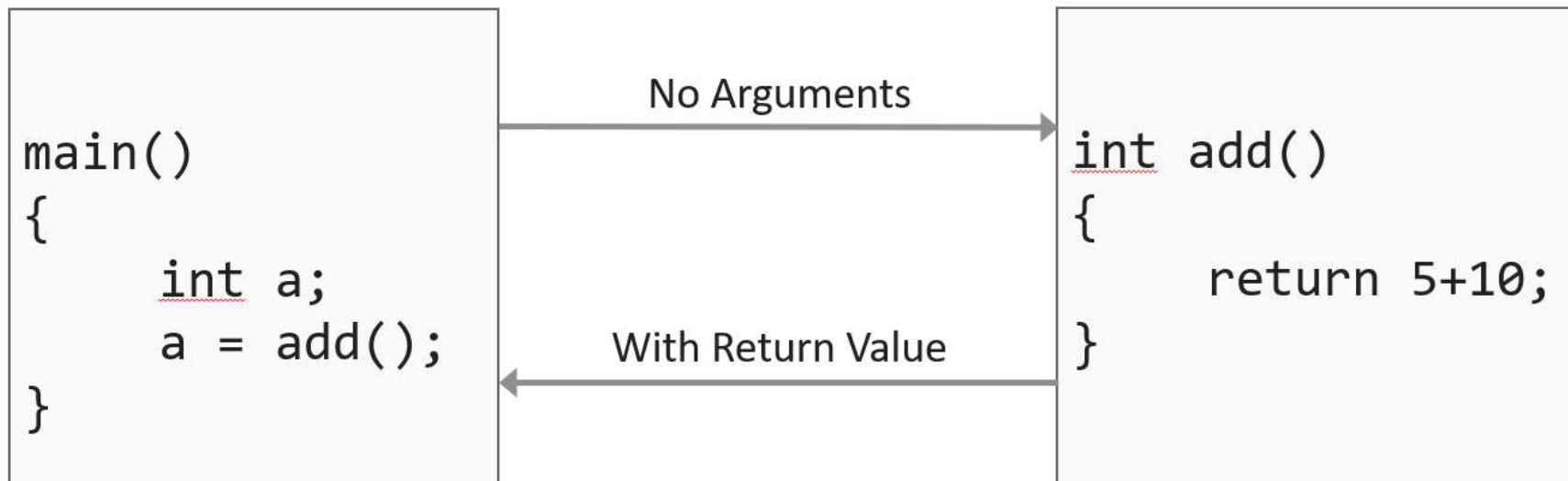
► Functions can be divided in 4 categories based on arguments and return value.

1. Method without arguments and without return value `void add();`
2. Method without arguments and with return value `int add();`
3. Method with arguments and without return value `void add(int, int);`
4. Method with arguments and with return value `int add(int, int);`

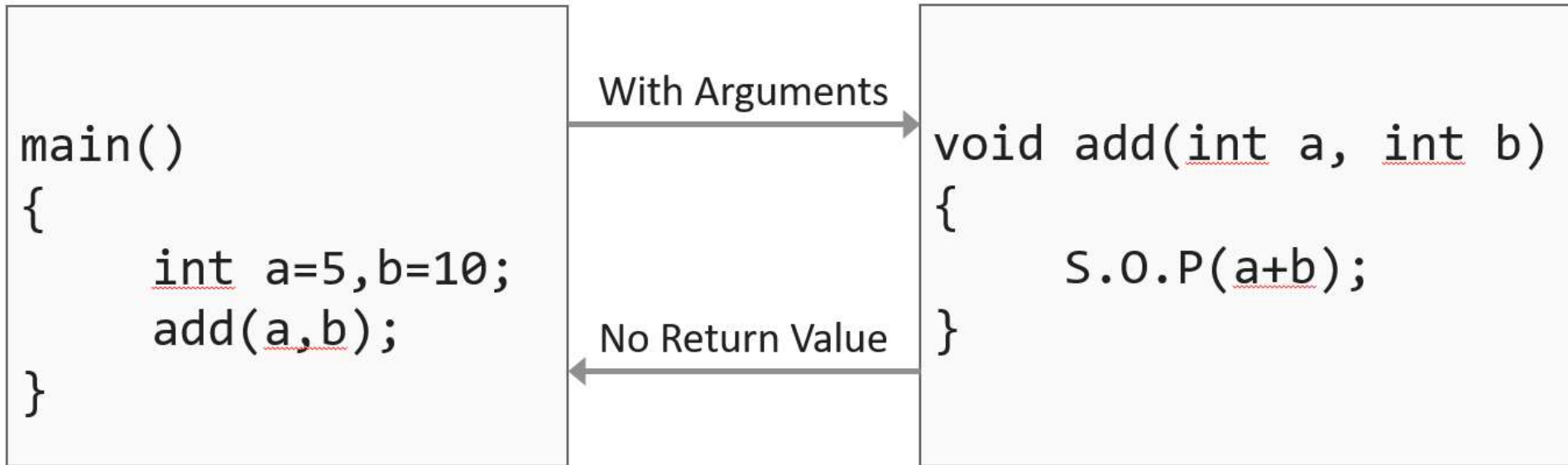
Method without arguments and without return value



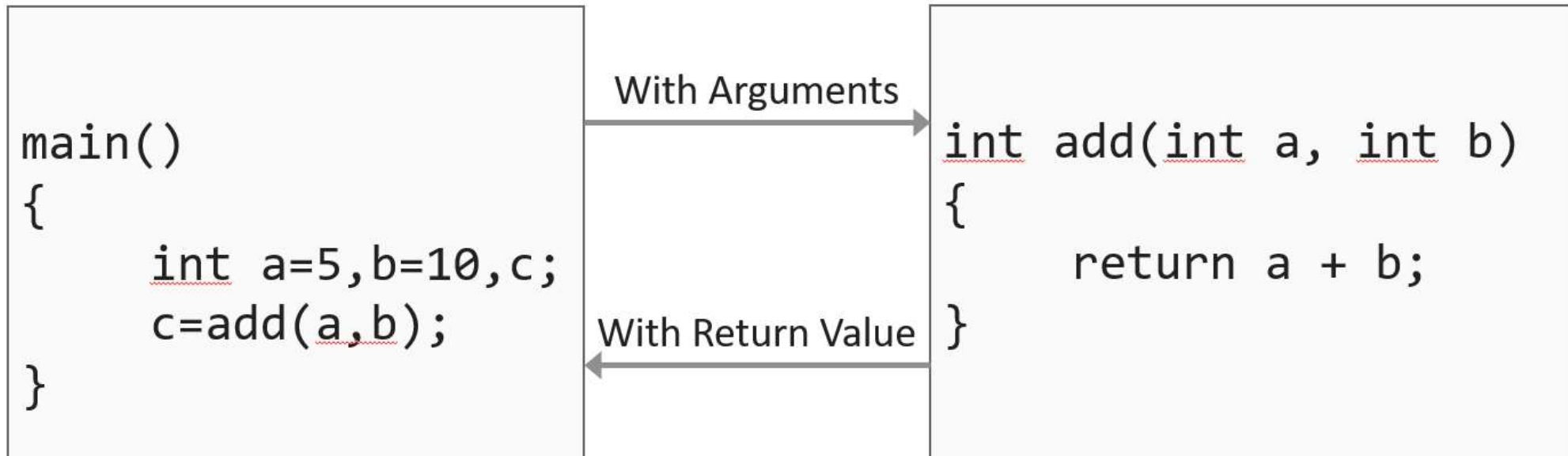
Method without arguments and with return value



Method with arguments and without return value



Method with arguments and with return value



Method Overloading: Compile-time Polymorphism

- ▶ **Definition:** When two or more methods are implemented that share same name but different parameter(s), the methods are said to be **overloaded**, and the process is referred to as **method overloading**
- ▶ Method overloading is one of the ways that Java implements polymorphism.
- ▶ When an overloaded method is invoked, Java uses the **type and/or number of arguments** as its guide to determine which version of the overloaded method to actually call.
 - ↳ E.g.

```
public void draw()  
public void draw(int height, int width)  
public void draw(int radius)
```
- ▶ Thus, overloaded methods must differ in the type and/or number of their parameters.
- ▶ While in overloaded methods with different return types and same name & parameter are not allowed, as the return type alone is **insufficient for the compiler** to distinguish two versions of a method.

Method Overloading: Compile-time Polymorphism

```
1. class Addition{
2.   int i,j,k;
3.   void add(int a){
4.     i=a;
5.     System.out.println("add i="+i);
6.   }
7.   void add(int a,int b){\\overloaded add()
8.     i=a;
9.     j=b;
10.    System.out.println("add i+j="+(i+j));
11.  }
12.  void add(int a,int b,int c){\\overloaded add()
13.    i=a;
14.    j=b;
15.    k=c;
16.    System.out.println("add i+j+k="+(i+j+k));
17.  }
18.}
```

```
19. class OverloadDemo{
20.   public static void
      main(String[] args){
21.     Addition a1= new Addition();
22.     //call all versions of add()
23.     a1.add(20);
24.     a1.add(30,50);
25.     a1.add(10,30,60);
26.   }
27.}
```

Output

```
t
add i=20
add i+j=80
add i+j+k=100
```


Method Overloading: Compile-time Polymorphism

```
1. class Addition{
2.   int i,j,k;
3.   void add(int a){
4.     i=a;
5.     System.out.println("add i="+i);
6.   }
7.   void add(int a,int b){\\overloaded add()
8.     i=a;
9.     j=b;
10.    System.out.println("add i+j="+(i+j));
11.  }
12.  void add(double a, double b){\\overloaded add()
13.    System.out.println("add a+b="+(a+b));
14.  }
15.  void add(int a,int b,int c){\\overloaded add()
16.    i=a;
17.    j=b;
18.    k=c;
19.    System.out.println("add i+j+k="+(i+j+k));
20.  }
21. }
```

```
22. class OverloadDemo{
23.   public static void
24.     main(String[] args){
25.     Addition a1= new Addition();
26.     //call all versions of add()
27.     a1.add(20);
28.     a1.add(30,50);
29.     a1.add(10,30,60);
30.     a1.add(30.5,50.67);
31.   }
```

Output

```
add i=20
add i+j=80
add i+j+k=100
add a+b=81.17
```


Method Overloading: Points to remember

- ▶ Method overloading supports polymorphism because it is one way that Java implements the “one interface, multiple methods” paradigm.
- ▶ Overloading **increases** the readability of the program.
- ▶ There are two ways to overload the method in java
 1. By changing number of **arguments**
 2. By changing the **data type**
- ▶ In java, method overloading is **not possible** by changing the **return type** of the method only because of **ambiguity**.

Method Overloading: Points to remember

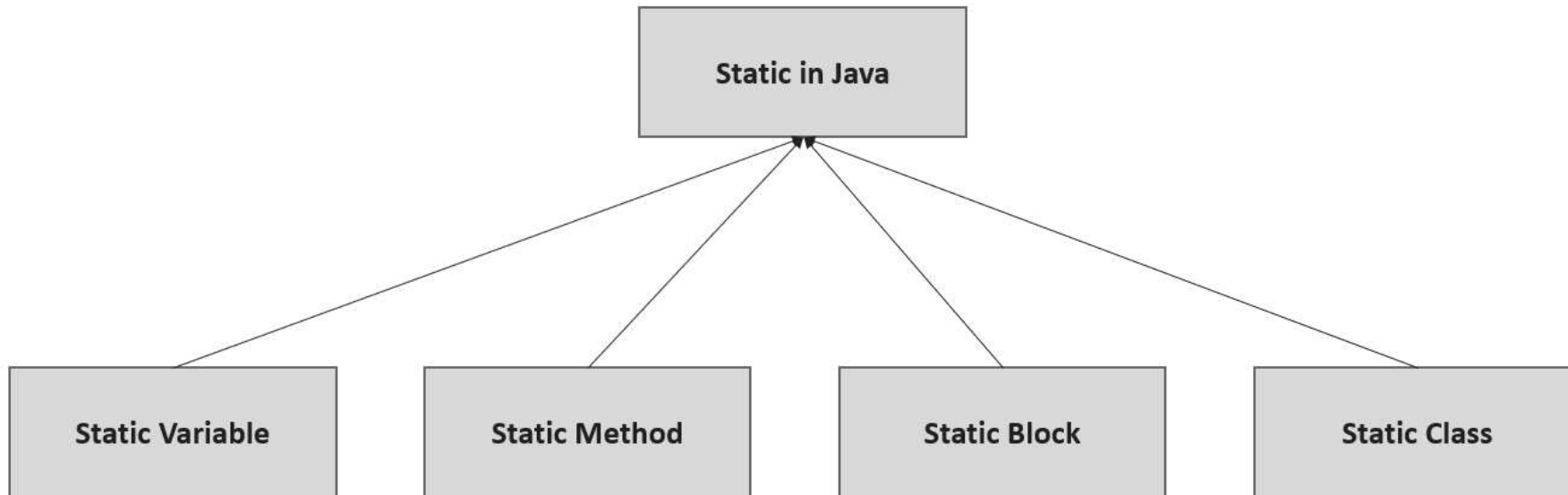
Can we overload java main() method?

- ▶ Yes, by method overloading. We can have any number of main methods in a class by method overloading
- ▶ But JVM calls main() method which receives string array as arguments only.

Advantages of Method

- ▶ Reduced Code **Redundancy**
 - ↳ Rewriting the same logic or code again and again in a program can be avoided.
- ▶ **Reusability** of Code
 - ↳ Same function can be call from multiple times without rewriting code.
- ▶ **Reduction** in size of program
 - ↳ Instead of writing many lines, just function need to be called.
- ▶ Saves Development **Time**
 - ↳ Instead of changing code multiple times, code in a function need to be changed.
- ▶ More **Traceability** of Code
 - ↳ Large program can be easily understood or traced when it is divide into functions.
- ▶ Easy to **Test & Debug**
 - ↳ Testing and debugging of code for errors can be done easily in individual function.

Static in java



Static

- ▶ The **static keyword** is used for **memory management**.
- ▶ We can apply static keyword with **variables, methods, blocks and nested classes**.
- ▶ The static keyword belongs to the class than an instance of the class.
- ▶ The static can be:
 1. Variable (also known as a class variable)
 2. Method (also known as a class method)
 3. Block
 4. Nested class

Static Variable

- ▶ Static variables have a property of **preserving** their **value** even after they are out of their **scope**.
- ▶ The static variable gets memory only once in the class area at the time of **class loading**.
- ▶ **Advantage of static variable:** It makes program **memory efficient** (static variable saves memory).
- ▶ **Characteristics of static variable:**
 - It is initialized to zero when the first object of its class is created. No other initialization is allowed.
 - **Only one copy** of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
 - It is visible only within the class, but its **lifetime** is the entire program.
 - Static variables are normally used to **maintain values** common for all objects.
 - The class constructor does not initialize static variable.

Static vs Non-Static Function

MyProgram.java(Non-static function)

```
1. public class MyProgram {
2. public static void
   main(String[] args) {
3.   int a=1,b=2,c;
4.   MyProgram mp=new
                                   MyProgram();
5.   c = mp.add(a,b);
6.   System.out.println(c);
   }//main
7. public int add(int i,int j)
8. {
9.     return i + j;
10. }
11. }//class
```

MyProgram.java(static function)

```
1. public class MyProgram {
2. public static void
   main(String[] args) {
3.   int a=1,b=2,c;
4.   c = add(a,b);
5.   System.out.println(c);
6. }//main

7. static int add(int i,int j)
8. {
9.     return i + j;
10. }//class
```


Characteristic of static method

- ▶ A static method can call only other static methods and can not call a non-static method from it.
- ▶ A static method can be accessed directly by the class name and doesn't need any object
- ▶ A static method cannot refer to "this" or "super" keywords in anyway

Static Method: WAP using class Rectangle and calculate area

```
1. import java.util.*;
2. class Rectangle{
3.     static float height;
4.     static float width;
5.     static void calArea() {
6.         System.out.println( "Area= "
                               +height*width);
7.     } //calArea()
8. } //class

13. class MyRectangle {
14.     public static void main(String[] args){
15.         Rectangle r1= new Rectangle();
16.         Scanner sc= new Scanner(System.in);
17.         System.out.print("enter height:");
18.         r1.height=sc.nextFloat();
19.         System.out.print("enter width:");
20.         r1.width=sc.nextFloat();
21.         Rectangle.calArea();
22.     } //main()
23. } //class
```

Output

```
enter height:30.55
enter width:20.44
```

```
Area=624.442
```

static keyword

- ▶ **static** keyword is mainly used for *memory management*.
- ▶ It can be used with
 - ↳ Variables
 - ↳ Methods
 - ↳ Blocks
 - ↳ Nested classes
- ▶ Basically, static is used for a constant variable or a method that is same for every instance of a class.
- ▶ The static variable can be used to refer to the common property of all objects.
- ▶ The static variable gets memory only once in the class area at the time of class loading.
- ▶ It makes your program memory efficient.
- ▶ Syntax
static type variablename;

```

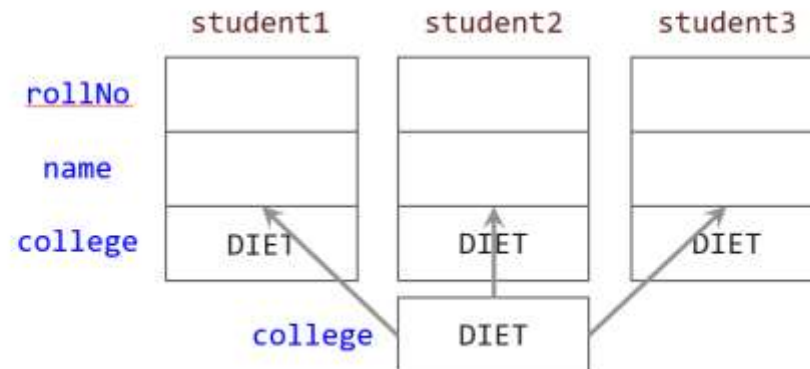
class Student {
    int rollNo;
    String name;
    String college;
    String collegeET#;"DIET";
    .
    .
    .
}

```

```

class StudentDemo {
    public static void main(String args[]) {
        Student student1 = new Student();
        Student student2 = new Student();
        Student student3 = new Student();
        .
        .
        .
    }
}

```



static method

- ▶ If you apply static keyword with any method, it is known as static method.
- ▶ A static method belongs to the **class** rather than the object of a class.
- ▶ A static method can be invoked without the need for creating an instance of a class.
- ▶ A static method can access static data member and can change the value of it.
- ▶ **Restrictions**
 1. The static method can not use non static data member or call non-static method directly.
 2. **this** and **super** cannot be used in static context.

```
class Student {
    int rollno;
    String name;
    static String college="abc";
    static void change() {
        college = "DIET";
        rollno = 10;
    }
    Student(int r, String n) {
        rollno = r;
        name = n;
    }
    void display() {
        System.out.println(rollno+" "+name+" "+college);
    }
}

class TestStaticMethod {
    public static void main(String args[]) {
        Student.change();
        Student s1 = new Student(111,"Tom");
        Student s2 = new Student(222,"Jerry");

        s1.display();
        s2.display();
    }
}
```

C:\Windows\System32\cmd.exe

```
D:\Java2021Demo>java TestStaticMethod
111 Tom abc
222 Jerry abc
```

We can not use **non-static** variables in **static methods**

C:\Windows\System32\cmd.exe

```
D:\Java2021Demo>java TestStaticMethod
111 Tom DIET
222 Jerry DIET
```


Static Block

- ▶ Static block is executed **exactly once**, when the class is first loaded.
- ▶ It is used to **initialize static variables** of the class.
- ▶ It will be executed even **before the main()** method.

▶ Syntax

```
static {  
    //initialisation of static variables...  
}
```

▶ **How to call static block in java?**

- ↳ Unlike method, there is no specified way to call a static block.
- ↳ The static block **executes automatically** when the class is loaded in memory.

Example: Static Block, Method and Variable

```
1. class StaticDemo {  
2.     static int a = 4;    //static variable declared & initial  
3.     static int b;       //static variable declared  
4.     static void dispValue(int x) {  
5.         System.out.println("Static method initialized.");  
6.         System.out.println("x = " + x);  
7.         System.out.println("a = " + a);  
8.         System.out.println("b = " + b);  
9.     } //static method  
10.    static {  
11.        System.out.println("Static block initialized.");  
12.        b = a * 5;  
13.    } //static block  
14.    public static void main(String args[]) {  
15.        System.out.println("inside main()...");  
16.        dispValue(44);  
17.    } //main()  
18.} //class
```

Output

```
Static block initialized.  
inside main()...  
Static method initialized.  
x = 44  
a = 4  
b = 20
```

Points to remember for static keyword

1. When we declare a field static, exactly a **single copy** of that field is created and **shared** among all instances of that class.
2. Static variables belong to a **class**, we can access them directly using class name. Thus, we don't need any object reference.
3. We can only **declare** static variables at the **class** level.
4. We can access static fields without **object initialization**.
5. Static methods can't be **overridden**.
6. **Abstract** methods can't be static.
7. Static methods can't use **this** or **super** keywords.
8. Static methods can't access instance variables and instance methods directly. They need some object reference to do so.
9. A class can have **multiple static** blocks.

